



JAIN
DEEMED-TO-BE UNIVERSITY

SCHOOL OF
COMPUTER
SCIENCE AND IT

Department of CS and IT

4th Semester

MOBILE COMPUTING

(23BCAM4C01)

Activity 2

Daily XP: A Mini Productivity Game

A Gamified Task Manager with SharedPreferences & XP Progression

SUBMITTED TO

Dr. RAMKUMAR KRISHNAMOORTHY

Submitted by:

Sooraj C-24BCAR0098

Welcome



CERTIFICATE

This is to certify that **Mr. Sooraj C** has satisfactorily completed **Activity-2** on 03/05/2026 prescribed by JAIN Deemed to be university for the **Fourth Semester** Degree course in the year 2026-2027.

S. No.	CRITERIA	MARKS	MARKS OBTAINED
1	Abstract	3	
2	Implementation	10	
3	Diagram	3	
4	Report/Views	4	
5	Viva	3	
6	On-Time Submission	2	
	TOTAL	25	

MARKS	
Total	Conversion of Marks to 15
25	

Signature of the Student

Signature of the Staff member

TABLE OF CONTENTS

SECTION	PAGE
1. Abstract & Introduction	4-5
2. Implementation Scenario in Real-Time	6-7
3. Architecture & System Design	8-9
4. Key Features & Modules	10-11
5. Diagram	12
6. Your Views & Concluding Remarks	13-15
7. References	16

1.ABSTRACT

Daily XP is a lightweight Android application that transforms the mundane act of managing a to-do list into an engaging mini role-playing game experience. Users add tasks to their daily list, and upon completion each task awards a configurable amount of Experience Points (XP). A visual progress bar displays the current XP level, and a daily reset button clears the slate at the end of each day. All user data — task list, accumulated XP, and level threshold — is persisted locally using Android SharedPreferences, making the application entirely offline and privacy-respecting.

The core objective of Daily XP is to leverage gamification principles — specifically XP-based progression found in RPG games — to motivate users to complete their daily tasks. By rewarding task completion with visible, quantifiable progress, the app addresses the common productivity challenge of procrastination and lack of motivation. This report documents the concept, design, implementation approach, and personal reflections on the development of Daily XP.

1.1 INTRODUCTION

Productivity applications have been a staple of the mobile ecosystem for over a decade, yet user engagement and consistent usage remain persistent challenges. Standard to-do list applications provide functionality but lack the emotional reward loop that keeps users coming back.

Gamification — the application of game-design elements in non-game contexts — has been shown to significantly improve user engagement, motivation, and habit formation.

Daily XP bridges this gap by embedding a simple RPG progression system directly inside a task manager. The application enables:

Users can add named tasks with individually assigned XP values to a daily task list.

Visual XP progress tracking through a level bar (progress bar) that fills as tasks are completed.

Level-up feedback when the accumulated XP crosses the daily threshold, providing a dopamine reward similar to RPG level-up mechanics.

A daily reset mechanism that clears all tasks and XP at the start of a new day, maintaining the habit-forming daily cycle.

This project aligns with the Mobile Computing course (23BCAM4C01) as it demonstrates core mobile development concepts including Activity lifecycle management, SharedPreferences for persistent local storage, RecyclerView with custom adapters, ProgressBar widget manipulation, and event-driven UI updates — all implemented in an Android Studio environment using Java.

2. IMPLEMENTATION SCENARIO IN REAL-TIME USAGES

Real-World Use Cases

The Daily XP application addresses genuine productivity challenges faced by students, professionals, and anyone seeking to build consistent daily habits. The following use cases illustrate practical deployment scenarios.

Use Case 1 — Student Daily Study Planner

A university student opens Daily XP at the start of their study day and adds tasks such as 'Complete OS assignment' (50 XP), 'Read 10 pages of DBMS textbook' (30 XP), and 'Solve 5 coding problems' (70 XP). As each task is checked off, the XP bar advances visibly toward the daily 200 XP goal. The gamified progression motivates the student to complete all tasks rather than abandoning the list mid-day — a common failure mode of traditional to-do apps.

Use Case 2 — Professional Task Management

A software developer uses Daily XP to manage sprint tasks. High-priority tickets are awarded more XP, naturally incentivising the completion of important work first. The daily reset at midnight mirrors the Agile daily standup cycle, ensuring each workday begins fresh without the psychological weight of accumulated incomplete tasks.

Use Case 3 — Habit Formation & Wellness

A user building healthy habits adds recurring daily tasks such as 'Drink 8 glasses of water' (20 XP), 'Exercise for 30 minutes' (60 XP), and 'Read before bed' (40 XP). The XP system provides tangible acknowledgement of small healthy choices, reinforcing the habit loop described in behavioral psychology. The local SharedPreferences storage means there is no account, no data sharing, and no dependency on internet connectivity.

Technology Stack

Technology	Usage
Android Studio	IDE for development, layout design, and emulator testing
Java	Primary programming language for all application logic
SharedPreferences	Local key-value storage for tasks, XP, and level state
RecyclerView	Efficient rendering of the dynamic task list
ProgressBar	Visual XP level bar with animated fill progression
CardView	Material Design cards for individual task items
AlertDialog	Popup dialog for adding new tasks with name and XP input
Gson (optional)	JSON serialization of task list objects for SharedPreferences

3.Architecture & System Design

Application Architecture

Daily XP follows a single-Activity architecture pattern with a clear separation of concerns across three logical layers. Given the lightweight nature of the application and the absence of a remote backend, the architecture is deliberately lean:

- **Presentation Layer:** XML layouts (activity_main.xml) containing the RecyclerView task list, ProgressBar XP indicator, level TextView, and action buttons. The MainActivity.java class manages all UI interactions.
- **Business Logic Layer:** Java methods within MainActivity handling XP calculation, level-up detection, task completion toggling, and daily reset logic.
- **Data Layer:** Android SharedPreferences acting as the sole persistence mechanism, storing the serialized task list (as JSON), current XP value, current level, and the last reset date for automatic daily reset detection.

Data Flow Diagram

The following describes the data flow within the Daily XP application:

- **App Launch:** MainActivity reads SharedPreferences to restore saved tasks, XP, and level.
- **Date Check:** If the stored reset date differs from today, an automatic daily reset is triggered.
- **Add Task:** User taps the '+' FAB → AlertDialog prompts for task name and XP value → task appended to RecyclerView and saved to SharedPreferences.
- **Complete Task:** User taps a task card checkbox → XP value added to current total → ProgressBar updates → level-up check triggered → state saved to SharedPreferences.
- **Level Up:** If $\text{currentXP} \geq \text{levelThreshold}$ → level counter increments → threshold scales up → toast or animation celebrates the level-up.
- **Daily Reset:** User taps Reset button (or auto-reset on new day) → task list cleared → XP reset to 0 → level reset to 1 → SharedPreferences updated.

SharedPreferences Data Structure

All application state is stored in a single SharedPreferences file ('dailyxp_prefs') using the following key-value schema:

Key	Type	Description
tasks_json	String	Gson-serialized JSON array of Task objects
current_xp	Integer	Accumulated XP for the current day
current_level	Integer	Current player level (starts at 1)
level_threshold	Integer	XP needed to reach next level (scales per level)
last_reset_date	String	ISO date string of last daily reset (yyyy-MM-dd)

4.KEY FEATURES & MODULES

Task Management with XP Assignment

The core feature of Daily XP is its task list with per-task XP values. When the user taps the Add Task FAB, an AlertDialog presents two input fields: the task name and the XP reward. Upon confirmation, the task is appended to the RecyclerView and immediately persisted to SharedPreferences. Each task card displays the task name, its XP value, and a checkbox. Checking the checkbox marks the task as complete, awards the XP, updates the progress bar, and saves the state — providing instant visual feedback.

XP Level Bar (Progress Bar)

The XP progression system is visualised through an Android ProgressBar widget styled to resemble an RPG experience bar. The progress is calculated as $(\text{currentXP} / \text{levelThreshold}) * 100$ and set via `setProgress()`. A companion TextView displays the numerical value (e.g., '120 / 200 XP'). The level threshold scales with each level-up, implementing a classic RPG curve where each subsequent level requires more XP, maintaining the challenge as the user becomes more productive.

Level-Up System

When accumulated XP reaches or exceeds the current level threshold, a level-up event is triggered: the current level increments, XP overflows carry forward, and the next threshold is calculated using a scaling formula (e.g., $\text{nextThreshold} = \text{currentThreshold} * 1.5$). A Toast notification celebrates the achievement. This feedback loop is the primary gamification mechanic, borrowing directly from RPG game design to create a sense of measurable progression.

Daily Reset

The daily reset feature maintains the habit-forming cycle of the application. A prominently placed Reset button clears all tasks, resets XP to zero, and returns the level to 1. Additionally, on each app launch, the current date is compared to the stored `last_reset_date` in SharedPreferences. If they differ, an automatic reset is performed silently, ensuring each new day begins fresh. This auto-reset mirrors the daily quest reset mechanic common in mobile RPGs and habit-tracking apps like Habitica.

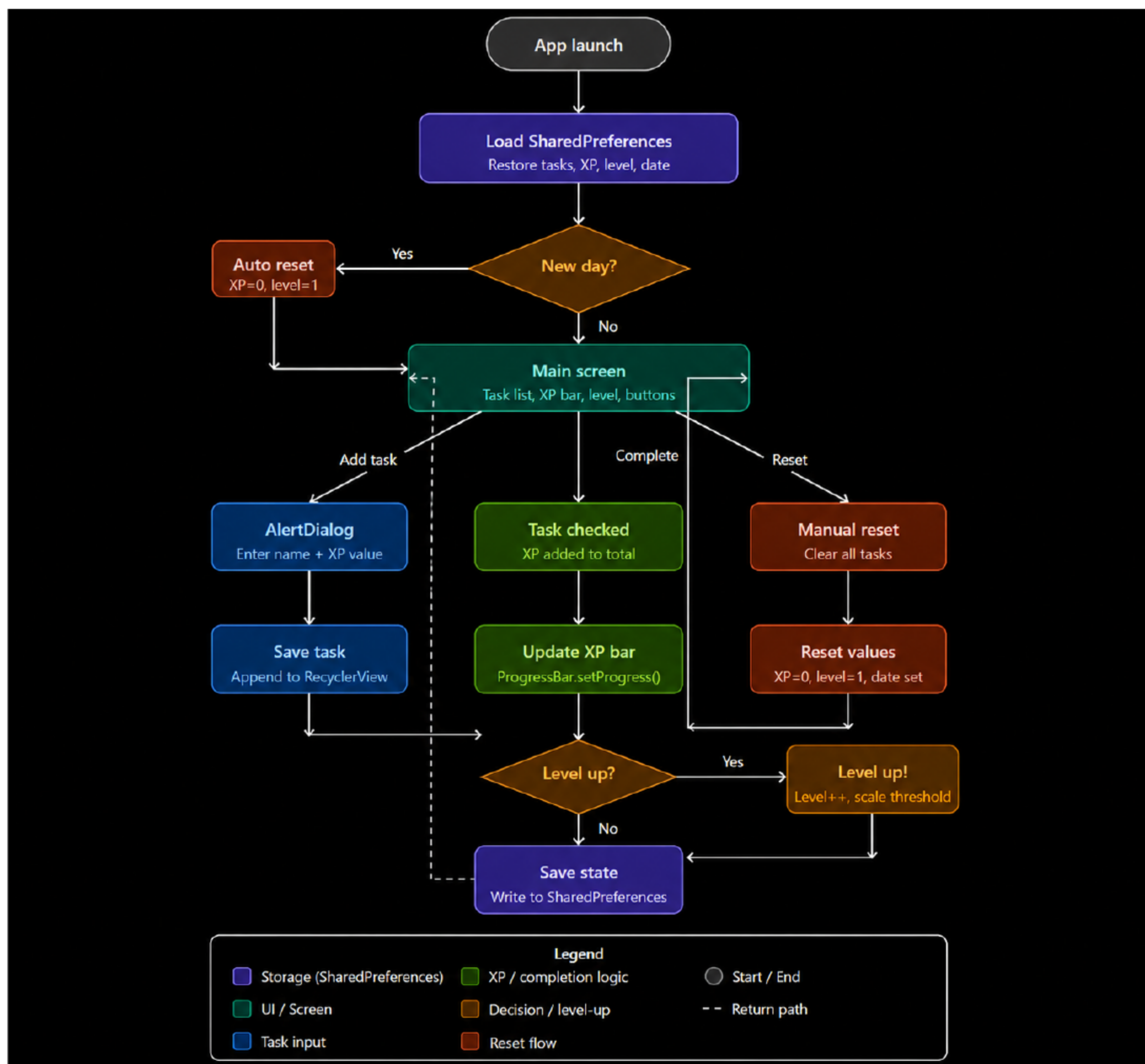
Persistent Local Storage via SharedPreferences

Unlike cloud-dependent applications, Daily XP stores all data locally using Android SharedPreferences. The task list — a complex object — is serialized to JSON using the Gson library before storage and deserialized on retrieval. This approach is entirely offline, requires no user account, and respects user privacy. The use of SharedPreferences also demonstrates a fundamental Android persistence pattern appropriate for lightweight, single-user data.

Activity & Module Summary

Activity / File	Responsibility
MainActivity.java	Central controller: task list, XP logic, level-up, reset, UI updates
TaskAdapter.java	RecyclerView adapter rendering task cards with checkbox completion
Task.java	Data model (POJO) for task name, XP value, and completion state
activity_main.xml	Main layout: RecyclerView, ProgressBar, level TextView, FAB, Reset button
item_task.xml	Individual task card layout with name, XP badge, and checkbox
SharedPreferencesHelper.java	Utility class wrapping SharedPreferences read/write operations

5.DIAGRAMS



6.VIEWS & CONCLUDING REMARKS

Learning Outcomes

Developing Daily XP provided meaningful exposure to practical Android development challenges beyond simple CRUD operations. The most significant learning was implementing SharedPreferences for complex data — specifically, understanding why primitive key-value storage requires serialization (via Gson) when storing lists of objects. This highlighted a fundamental trade-off in mobile storage: SharedPreferences is simple and fast for primitives but requires additional serialization overhead for complex types, whereas SQLite or Room provides structural querying at the cost of greater setup complexity.

Implementing the ProgressBar as a dynamic, real-time feedback mechanism reinforced the importance of immediate UI responsiveness in mobile UX. Every task completion updates the bar instantly, and achieving this without lag requires understanding the Android main thread and ensuring all UI operations are executed on it.

Challenges Faced

The primary technical challenge was correctly implementing the daily auto-reset logic. Storing and comparing dates using string formatting introduced timezone edge cases — a device changing timezone could potentially trigger unintended resets. The solution involved using Calendar.getInstance() with a consistent locale and formatting date strings with Locale.US to ensure deterministic comparison regardless of device locale settings.

A secondary challenge was designing the XP scaling formula to feel both achievable and progressively challenging. Early iterations used a linear scale that felt too easy; the final exponential multiplier (threshold $\times 1.5$ per level) provides a curve that balances short-term achievability with long-term depth, mirroring the progression curves studied in game design literature.

Innovative Aspects

Daily XP innovates by applying RPG game mechanics — specifically the XP and levelling system — to the traditionally dry domain of task management. Unlike existing gamified productivity apps such as Habitica (which require account creation and internet connectivity), Daily XP operates entirely locally, making it instantly usable without onboarding friction. The per-task XP assignment is a deliberate design choice: rather than all tasks being equal, users can reflect the relative importance of tasks through XP weighting, effectively creating a priority system disguised as a reward system.

Future Enhancements

- **Achievement Badges:** Award special badges (stored in `SharedPreferences`) for streaks such as 'Completed 7 days in a row' or '500 total XP earned'.
- **XP History Chart:** Display a weekly bar chart showing daily XP earned, implemented with `MPAndroidChart` library.
- **Task Categories:** Colour-code tasks by category (Work, Study, Health) with category-specific XP multipliers.
- **Widget Support:** Implement a home screen widget showing the current XP progress bar for at-a-glance motivation.
- **Cloud Sync (Optional):** Add an optional `Firebase Firestore` sync for users who want cross-device history, while maintaining offline-first functionality.

Conclusion

The Daily XP application successfully demonstrates the practical application of core Android development skills in a creative, user-centric context. The project covers key Mobile Computing topics including Android Activity lifecycle, RecyclerView with custom adapters, ProgressBar widget management, SharedPreferences-based local persistence, JSON serialization with Gson, and event-driven UI design.

More broadly, this activity illustrated how mobile technology can reframe mundane daily routines — in this case, completing a to-do list — into genuinely engaging experiences through thoughtful application of game design principles. The simplicity of the implementation (a single Activity, no backend, no account required) demonstrates that impactful user experiences do not necessarily demand complex infrastructure; they demand careful attention to human motivation and feedback design.

6. REFERENCES

1. Google LLC. *Firebase Documentation — Add Firebase to your Android Project*. Firebase for Android. Available at: <https://firebase.google.com/docs/android/setup> [Accessed: May 2026]
2. Google LLC. *Cloud Firestore Documentation — Get Started with Cloud Firestore on Android*. Firebase Developer Documentation. Available at: <https://firebase.google.com/docs/firestore/quickstart> [Accessed: May 2026]
3. Google LLC. *Firebase Authentication — Authenticate with Firebase using Password-Based Accounts on Android*. Firebase Developer Documentation. Available at: <https://firebase.google.com/docs/auth/android/password-auth> [Accessed: May 2026]
4. Android Developers. *RecyclerView — Create dynamic lists with RecyclerView*. Android Developer Documentation. Available at: <https://developer.android.com/develop/ui/views/layout/recyclerview> [Accessed: May 2026]
5. Android Developers. *Introduction to Activities — Android Developer Fundamentals*. Android Developer Documentation. Available at: <https://developer.android.com/guide/components/activities/intro-activities> [Accessed: May 2026]
6. Android Developers. *Connect your app to Firebase using Android Studio*. Android Developer Documentation. Available at: <https://developer.android.com/studio/write/firebase> [Accessed: May 2026]



Thank
YOU